

MongoDB 與 Python 全端開發完整指南

基於 Docker Compose 與 MongoDB Compass

開發者教學手冊

2026年7月20日

目录

1 基礎環境建置：Docker Compose 與 MongoDB	2
1.1 為什麼選擇 Docker 部署 MongoDB?	2
1.2 撰寫 docker-compose.yml	2
1.3 啟動與驗證	3
2 強大的視覺化工具：MongoDB Compass	4
2.1 安裝與連線	4
2.1.1 取得連線字串 (Connection String)	4
2.1.2 連線步驟	4
2.2 Compass 核心功能導覽	4
3 MongoDB 核心基礎與指令 (CLI)	5
3.1 資料庫與集合 (Database & Collection)	5
3.2 CRUD 基礎操作	5
3.2.1 Create (新增)	5
3.2.2 Read (讀取)	5
3.2.3 Update (更新)	6
3.2.4 Delete (刪除)	6
4 Python 整合與開發：PyMongo 完整教學	7
4.1 安裝與初始化	7
4.2 建立資料庫連線	7
4.3 Python 中的 CRUD 操作	7
4.3.1 新增資料 (Insert)	7
4.3.2 查詢資料 (Find)	8
4.3.3 更新資料 (Update)	8
4.3.4 刪除資料 (Delete)	8
5 進階應用：聚合 (Aggregation) 與索引	9
5.1 為什麼需要 Aggregation?	9
5.2 Python 中的 Aggregation 實作	9
5.3 建立索引 (Indexing) 提升效能	9
6 結語與下一步	11

第 1 章

基礎環境建置：Docker Compose 與 MongoDB

1.1 為什麼選擇 Docker 部署 MongoDB?

在現代軟體開發中，使用 Docker 容器化技術部署資料庫已經成為業界標準。它可以確保「開發、測試、正式環境」的一致性，並且不需在主機上安裝繁雜的資料庫依賴套件。透過 docker-compose，我們甚至可以一鍵啟動資料庫與相關的管理工具。

1.2 撰寫 docker-compose.yml

以下是建立 MongoDB 以及視覺化管理工具 Mongo Express 的完整設定檔。請建立一個名為 docker-compose.yml 的檔案並貼入以下內容：

```
1 version: '3.8'
2
3 services:
4   mongodb:
5     image: mongo:latest
6     container_name: mongodb_server
7     restart: always
8     ports:
9       - "27017:27017"
10    environment:
11      MONGO_INITDB_ROOT_USERNAME: admin
12      MONGO_INITDB_ROOT_PASSWORD: password123
13    volumes:
14      - mongo_data:/data/db
15
16   mongo-express:
17     image: mongo-express:latest
18     container_name: mongo_express_ui
19     restart: always
20     ports:
21       - "8081:8081"
22     environment:
23       ME_CONFIG_MONGODB_ADMINUSERNAME: admin
24       ME_CONFIG_MONGODB_ADMINPASSWORD: password123
25       ME_CONFIG_MONGODB_SERVER: mongodb
26     depends_on:
27       - mongodb
28
```

```
29 volumes:
30   mongo_data:
```

1.3 啟動與驗證

在終端機中，切換到該檔案所在的目錄，並執行以下指令啟動服務：

```
1 docker-compose up -d
```

啟動後，MongoDB 將運行於本地的 27017 埠。您可以透過瀏覽器前往 <http://localhost:8081> 瀏覽 Mongo Express 的簡易網頁介面，確認資料庫是否正常運作。

第 2 章

強大的視覺化工具：MongoDB Compass

2.1 安裝與連線

MongoDB Compass 是官方提供的最強大桌面端 GUI 工具，適合用來進行資料探索、查詢效能優化與建立 Aggregation Pipeline。

2.1.1 取得連線字串 (Connection String)

基於我們第一章的 Docker 設定，連線字串的格式為 `mongodb:// : @ :` 。我們的字串將是：

```
1 mongodb://admin:password123@localhost:27017
```

2.1.2 連線步驟

1. 下載並開啟 MongoDB Compass。
2. 在主畫面的 "New Connection" 輸入框中，貼上上述連線字串。
3. 點擊 "Connect" 按鈕。連線成功後，您將看到預設的資料庫列表 (如 admin, config, local)。

2.2 Compass 核心功能導覽

- **Documents 分頁：**這是您最常使用的畫面。您可以直接看到資料 (JSON 文件形式)，點擊右側的筆型圖示或雙擊資料，即可直接進行修改，無需撰寫任何指令。
- **Aggregations 分頁：**這是 Compass 的殺手級功能。如果您需要撰寫複雜的資料聚合管線 (如分組、加總、關聯)，這裡提供了「所見即所得」的區塊式編輯器，幫助您逐步驗證每一層處理後的資料樣貌。
- **Explain Plan 分頁：**當您的查詢速度變慢時，這個分頁能幫您分析查詢語句是否有效利用了索引 (Index)，是資料庫效能優化的重要工具。
- **Indexes 分頁：**您可以在此處視覺化地新增、刪除索引。

第 3 章

MongoDB 核心基礎與指令 (CLI)

在學習程式語言串接之前，了解 MongoDB 原生的操作指令是必須的。您可以透過終端機進入 MongoDB 容器來練習：

```
1 docker exec -it mongodb_server mongosh -u admin -p password123
```

3.1 資料庫與集合 (Database & Collection)

MongoDB 屬於 NoSQL 文件型資料庫。關聯式資料庫中的「資料表 (Table)」在這邊稱為「集合 (Collection)」，「列 (Row)」稱為「文件 (Document)」。

- 顯示所有資料庫：show dbs
- 切換或建立資料庫：use myAppDB (如果資料庫不存在，在插入第一筆資料時會自動建立)
- 顯示所有集合：show collections

3.2 CRUD 基礎操作

3.2.1 Create (新增)

新增單筆或多筆文件到 users 集合中。

```
1 //
2 db.users.insertOne({ name: "Alice", age: 25, role: "developer" })
3
4 //
5 db.users.insertMany([
6   { name: "Bob", age: 30, role: "manager" },
7   { name: "Charlie", age: 35, role: "developer" }
8 ])
```

3.2.2 Read (讀取)

尋找符合條件的資料。MongoDB 使用 JSON 格式來定義查詢條件。

```
1 //
2 db.users.find()
3
4 //      role    developer
5 db.users.find({ role: "developer" })
```

```
6
7 //      age    (gt) 28
8 db.users.find({ age: { $gt: 28 } })
```

3.2.3 Update (更新)

更新分為 updateOne 和 updateMany。必須使用 \$set 運算符來指定要更新的欄位。

```
1 //  Alice    26
2 db.users.updateOne(
3   { name: "Alice" }, //
4   { $set: { age: 26 } } //
5 )
```

3.2.4 Delete (刪除)

```
1 //  Bob
2 db.users.deleteOne({ name: "Bob" })
```

第 4 章

Python 整合與開發：PyMongo 完整教學

Python 搭配 MongoDB 是資料科學、爬蟲開發與網頁後端 (如 FastAPI, Flask) 的絕佳組合。官方提供的驅動程式為 PyMongo。

4.1 安裝與初始化

請先在您的 Python 環境中安裝套件：

```
1 pip install pymongo
```

4.2 建立資料庫連線

以下是在 Python 中連接至我們剛才用 Docker 建立的 MongoDB 伺服器的標準做法。

```
1 from pymongo import MongoClient
2
3 #
4 CONNECTION_STRING = "mongodb://admin:password123@localhost:27017"
5
6 # MongoClient
7 client = MongoClient(CONNECTION_STRING)
8
9 #
10 db = client['python_tutorial_db']
11
12 # (Collection)
13 users_collection = db['users']
14
15 print(" ")
```

4.3 Python 中的 CRUD 操作

4.3.1 新增資料 (Insert)

```
1 #
2 user_1 = {
3     "name": "David",
4     "email": "david@example.com",
5     "skills": ["Python", "Docker", "MongoDB"]
```



```

6 }
7 result = users_collection.insert_one(user_1)
8 print(f"    ID: {result.inserted_id}")
9
10 #
11 users = [
12     {"name": "Eva", "email": "eva@example.com", "age": 28},
13     {"name": "Frank", "email": "frank@example.com", "age": 32}
14 ]
15 results = users_collection.insert_many(users)
16 print(f"    {len(results.inserted_ids)}    ")

```

4.3.2 查詢資料 (Find)

```

1 #
2 print("---    ---")
3 for user in users_collection.find():
4     print(user)
5
6 #    ( age    30    )
7 print("---    30    ---")
8 query = {"age": {"$gt": 30}}
9 for user in users_collection.find(query):
10     print(user["name"], user["age"])

```

4.3.3 更新資料 (Update)

```

1 #    Eva    skills
2 query_filter = {"name": "Eva"}
3 update_operation = {
4     "$set": {"skills": ["Design", "UI/UX"]},
5     "$inc": {"age": 1} #    1
6 }
7
8 result = users_collection.update_one(query_filter, update_operation)
9 print(f"    {result.matched_count}    {result.modified_count}    ")

```

4.3.4 刪除資料 (Delete)

```

1 #    Frank
2 delete_result = users_collection.delete_one({"name": "Frank"})
3 print(f"    {delete_result.deleted_count}    ")

```

第 5 章

進階應用：聚合 (Aggregation) 與索引

5.1 為什麼需要 Aggregation?

當單純的 `find()` 查詢無法滿足需求（例如：計算平均值、跨集合關聯查詢、資料分組統計）時，就需要使用 Aggregation Pipeline。它就像是一個工廠的輸送帶，資料依序經過多個階段 (Stage) 的處理。

5.2 Python 中的 Aggregation 實作

假設我們有許多產品銷售紀錄，我們想要計算「每種商品」的「總銷售額」。

```
1 # sales_collection
2 pipeline = [
3     # 1 "completed"
4     {"$match": {"status": "completed"}},
5
6     # 2 "item" ( * )
7     {"$group": {
8         "_id": "$item",
9         "totalRevenue": {
10             "$sum": {"$multiply": ["$quantity", "$price"]}
11         },
12         "totalSold": {"$sum": "$quantity"}
13     }},
14
15     # 3
16     {"$sort": {"totalRevenue": -1}}
17 ]
18
19 results = sales_collection.aggregate(pipeline)
20
21 for row in results:
22     print(f" : {row['_id']}, : {row['totalRevenue']}, : {row['totalSold']}")
```

5.3 建立索引 (Indexing) 提升效能

當資料量達到百萬級別時，沒有索引的查詢會導致「全表掃描 (CollScan)」，非常消耗效能。在 Python 中可以輕鬆建立索引：

```
1 import pymongo
2
```

```
3 # email (Unique)
4 users_collection.create_index(
5     [("email", pymongo.ASCENDING)],
6     unique=True
7 )
8 print(" ")
```

建立索引後，您可以回到 MongoDB Compass 的 Indexes 分頁，確認索引是否已經生效。

第 6 章

結語與下一步

恭喜您！這份手冊帶您走過了從基礎架設到進階程式串接的完整流程。MongoDB 的靈活 Schema 設計賦予了極大的開發彈性。

下一步建議：

- 嘗試將 PyMongo 整合到 FastAPI 或 Flask 等 Web 框架中，打造完整的 RESTful API 後端。
- 深入研究 ODM (Object-Document Mapper) 工具，例如 Python 的 MongoEngine 或 Beanie (非同步支援)，這將讓您的程式碼更具物件導向風格。
- 在 Compass 中多練習 Aggregation 區塊，將其自動轉譯為 Python 代碼，這能大幅節省開發時間。